

Smart Parking Camera Project Report

Group Members

- Medvegy Gábor
- Medvedev Matvey
- Anna Miha
- Tursynbay Zhaniya

1. Introduction

The goal of our project is to detect whether a parking space is **Empty** or **Occupied** from images. Our solution is a binary image classification task with practical real-world relevance. With the required resources, this model could be deployed to be used in intelligent parking systems, traffic optimization, and parking guidance applications.

The model we built receives an image of a single parking slot and outputs one of two classes:

- **Empty**
- **Occupied**

The main challenge was not defining the target label, but building a pipeline that remains accurate under different conditions such as changing weather, different parking sites, and variations in lighting and viewpoint.

We implemented a full end-to-end workflow, which consisted of the following steps:

- dataset indexing and split generation
- preprocessing and augmentation
- model training
- evaluation
- robustness analysis
- visual system-level output

The original PKLot segmented parking-slot dataset consists of around 700,000 labeled parking-slot images, while our available development machines were CPU-only. Because of this, the final reported experiment was carried out on a reproducible subset that preserved the 70/15/15 train/validation/test split. The final subset was also semi-stratified by parking site and occupancy class to keep it more representative.

2. Problem Formulation

The problem can be formalized as a supervised binary classification problem.

Given an input image x representing a single parking slot, the model predicts a label y :

- $y = 0$ for **Empty**

- $y = 1$ for Occupied

The objective of training is to learn a function:

$f(x) \rightarrow \{\text{Empty}, \text{Occupied}\}$

that generalizes well across different parking lots and environmental conditions.

Our project deliberately focuses on already segmented parking-slot images rather than full-scene parking-lot analysis. This means the system does not need to detect slot coordinates in the full camera image. Instead, it solves the classification step directly.

This design choice also makes the project easier to evaluate quantitatively, because every sample already has a clean ground-truth label.

3. Dataset

3.1 Source dataset

The project uses the PKLot parking occupancy dataset stored locally in `PKLot.tar`. The dataset contains images from multiple parking locations and weather conditions. In this project, the segmented parking-slot images were used, because they are already labeled and directly suited for supervised learning.

Each sample is associated with metadata extracted from the folder structure:

- parking site
- weather condition
- date
- class label

This metadata was later used for robustness analysis.

3.2 Full dataset organization

With the dataset preparation script, we first indexed the full set of segmented images and created split metadata files. The complete split sizes are:

Split	Count	Empty	Occupied
Train	487,093	250,648	236,445
Validation	104,374	53,709	50,665
Test	104,384	53,714	50,670

The split ratios used at this stage were:

- train: 70%
- validation: 15%
- test: 15%

3.3 Final experimental subset

Because we considered the full dataset too large for CPU-only experimentation, in the final reported experiment we used a reproducible subset while preserving the original 70/15/15 split ratio.

The final subset sizes are:

Split	Count used in final experiment
Train	2,100
Validation	450
Test	450

The total size of the subset was 3000 images.

The subset was sampled reproducibly with a fixed random seed, and the final subset selection was semi-stratified by parking site and occupancy class. This means the final sample better reflects the original split structure than a purely random subset would.

4. Data Preparation and Preprocessing

4.1 Metadata generation

In the first stage of the pipeline we scan the PKLot archive and generate CSV metadata files for the train, validation, and test splits. These CSV files store:

- archive member path
- class label
- numeric label ID
- site
- weather
- date
- filename

This makes the dataset handling reproducible and decouples data indexing from training.

4.2 Optional extraction

In the project we support two possible loading modes:

1. reading images directly from the tar archive
2. reading extracted images from a normal directory structure

Direct tar reading is memory-efficient, but slower. For the final experiment, the extracted files were used because they provide faster training and evaluation on a local CPU environment.

4.3 Image preprocessing

Every parking-slot image is preprocessed in the following way:

- resize to 128 x 128 so that all images have the same input size for the CNN
- convert to RGB to ensure a consistent three-channel color representation
- normalize pixel values to [0, 1] to scale the raw image intensities into a stable numeric range
- apply channel-wise normalization using standard mean and standard deviation to make training more stable and improve convergence

This produces a consistent input tensor for the CNN.

4.4 Data augmentation

To reduce overfitting and improve generalization, we applied data augmentation during training. This augmentation creates slightly modified versions of the training images, which help the model become less sensitive to small visual changes and prevents it from memorizing the exact appearance of the training set. We used the following transformations:

- random horizontal flip to expose the model to small viewpoint variations
- small random rotation (+-10 degrees) to simulate small camera angle differences
- brightness jitter to make the model more robust to lighting changes
- contrast jitter to help the model handle differences in image clarity and illumination

These transformations were intentionally small. Strong augmentation could damage the semantic meaning of a parking-slot image, while light augmentation improves robustness under realistic visual variations such as shadows, lighting shifts, and minor viewpoint changes.

5. Model Architecture

The baseline model is a custom convolutional neural network designed to remain lightweight and trainable on CPU.

5.1 Structure

The implemented model contains:

- 4 convolutional layers with channel sizes 3 -> 32 -> 64 -> 128 -> 256
- batch normalization after each convolution
- ReLU nonlinearities
- max-pooling after the first three convolutional blocks for progressive spatial downsampling
- adaptive average pooling
- a fully connected classifier head with dimensions 256 -> 128 -> 2
- dropout regularization

In code, the feature extractor is:

1. `Conv2d(3, 32, kernel_size=3, padding=1)`
2. `BatchNorm2d(32)`
3. `ReLU(inplace=True)`

4. `MaxPool2d(2)`
5. `Conv2d(32, 64, kernel_size=3, padding=1)`
6. `BatchNorm2d(64)`
7. `ReLU(inplace=True)`
8. `MaxPool2d(2)`
9. `Conv2d(64, 128, kernel_size=3, padding=1)`
10. `BatchNorm2d(128)`
11. `ReLU(inplace=True)`
12. `MaxPool2d(2)`
13. `Conv2d(128, 256, kernel_size=3, padding=1)`
14. `BatchNorm2d(256)`
15. `ReLU(inplace=True)`
16. `AdaptiveAvgPool2d((1, 1))`

The classifier head is:

1. `Flatten()`
2. `Linear(256, 128)`
3. `BatchNorm1d(128)`
4. `ReLU(inplace=True)`
5. `Dropout(p=0.3)`
6. `Linear(128, 2)`

Shortly, the model works as follows:

1. extract low-level visual features such as edges and texture
2. gradually build more abstract representations
3. compress the spatial information into a compact feature vector
4. classify the feature vector into `Empty` or `Occupied` categories

5.2 Why this model was chosen

We chose this model for our project because of the following:

- simple to explain
- fast enough for CPU-only training
- easy to debug
- fully controlled implementation
- sufficient performance for the assignment scope

6. Training Procedure

6.1 Hyperparameters

In the final reported experiment we used:

- optimizer: Adam
- learning rate: 0.001
- weight decay: 0.0001
- batch size: 64
- maximum epochs: 5
- early stopping patience: 2

6.2 Loss and output

The model outputs 2 logits, one for each class. During training, `CrossEntropyLoss` was used. This is appropriate for mutually exclusive class labels in binary classification when implemented with two output neurons.

6.3 Model selection

At the end of each epoch, the model was evaluated on the validation set. The best checkpoint was selected based on validation F1 score, not only raw accuracy. This is good because F1 balances precision and recall, and it is more informative than accuracy alone when discussing classification quality.

6.4 Early stopping

If validation performance stopped improving, early stopping prevents unnecessary extra training. In our experiment, the model completed the planned 5 epochs and the best checkpoint was at epoch 5.

7. Results

7.1 Validation performance during training

The final experiment produced the following validation metrics:

Epoch	Validation Accuracy	Validation F1
1	0.9422	0.9430
2	0.9800	0.9798
3	0.9822	0.9816
4	0.9822	0.9818
5	0.9844	0.9841

Best validation result:

- best epoch: 5
- best validation F1: 0.9841

The progression across epochs shows that the model learned quickly. Most of the improvement was in the first few epochs, after which performance stabilized at a very high level.

7.2 Final test performance

The final test performance of the best checkpoint was:

Metric	Value
Loss	0.0587
Accuracy	0.9867
Precision	0.9775
Recall	0.9954
F1 score	0.9864

These are strong results.

7.3 Interpretation

The results suggest:

- the chosen features are highly informative for this task
- the binary distinction between empty and occupied parking spaces is learnable with a relatively simple CNN
- the training pipeline is stable and learns meaningful patterns, not random noise

The very high recall shows that the model is especially good at identifying occupied spots. Precision is also high, showing relatively few false positives.

8. Robustness Analysis

8.1 Motivation

A single overall test accuracy does not fully describe model behavior. A classifier can look strong on average while performing poorly under certain environmental conditions. To address this, we included robustness analysis across:

- parking site
- weather condition

8.2 Results by site

Site	Count	Accuracy	F1
PUC	274	0.9927	0.9921
UFPR04	69	0.9565	0.9508
UFPR05	107	0.9907	0.9921

Interpretation:

- PUC and UFPR05 were especially strong in the chosen test subset
- UFPR04 was the hardest site-specific group
- even the weakest site-specific result remained strong

8.3 Results by weather

Weather	Count	Accuracy	F1
Cloudy	138	1.0000	1.0000
Rainy	75	1.0000	1.0000
Sunny	237	0.9747	0.9766

Interpretation:

- performance under Cloudy and Rainy conditions was perfect in this subset
- Sunny conditions were slightly more difficult
- the model still remained robust across all observed weather groups

8.4 What robustness means here

In this project, robustness means that the model does not only perform well on average, but also behaves reliably across different environments. That is important because a practical parking system should not fail just because weather or location changes.

At the same time, the robustness conclusions should still be interpreted with care, because the final experiment uses a subset rather than the full dataset.

9. System-Level Output

Beyond numerical metrics, we also created a simple visual demo that shows how the classifier could be used inside a parking-monitoring application.

The demo:

- samples parking-slot images
- predicts Empty or Occupied
- color-codes the predicted state
- displays confidence values
- reports the overall occupancy ratio for the displayed board

This output shows how the model we built could be used as real application component.

Generated artifact:

- `outputs/baseline_subset_70_15_15/system_output/occupancy_board.png`

This is not a full production system, because it operates on already segmented parking-slot images instead of a live full-scene feed. However, it is a meaningful system-level demonstration for the project scope.

10. Limitations

This project has limitations that need be stated.

10.1 Subset-based final training

The final reported experiment was performed on a reproducible subset, not on the full PKLot dataset. This was necessary due to hardware constraints, but it means the final metrics should be interpreted as strong subset-based baseline results rather than definitive full-dataset performance.

10.2 CPU-only environment

The development machine did not provide GPU acceleration, which limited the scale of experiments and made larger runs slower and less practical.

10.3 Segmented slot assumption

The pipeline assumes that the parking-slot crops are already available. It does not solve full-scene slot localization or multi-object detection from a live camera feed.

10.4 Baseline architecture

Although the chosen CNN performed very well, stronger architectures are possible. A pretrained transfer-learning backbone could likely improve robustness further, especially under more diverse conditions.

11. Conclusion

We successfully implemented a complete parking occupancy detection workflow:

- dataset preparation
- preprocessing and augmentation
- CNN-based binary classification
- validation and test evaluation
- robustness analysis
- visual system output

Even with a CPU-friendly architecture and a subset-based final experiment, we achieved:

- 98.67% test accuracy
- 0.9864 test F1 score

These results show that parking occupancy detection can be solved effectively with a relatively simple deep learning approach, provided that the input parking slots are already segmented.

So our project meets the practical goals of the assignment while remaining computationally realistic on local hardware.

12. Reproducibility

Main files used in the project:

- scripts/prepare_dataset.py
- scripts/extract_dataset.py

- `scripts/train_baseline.py`
- `scripts/robustness_analysis.py`
- `scripts/system_output_demo.py`
- `configs/baseline_subset_70_15_15.yaml`

Main outputs:

- `outputs/baseline_subset_70_15_15/metrics.json`
- `outputs/baseline_subset_70_15_15/robustness/summary.json`
- `outputs/baseline_subset_70_15_15/robustness/by_site.csv`
- `outputs/baseline_subset_70_15_15/robustness/by_weather.csv`
- `outputs/baseline_subset_70_15_15/system_output/occupancy_board.png`